ohhh so thats what i put first and your saying don't put the same thing at the very end

YESSS, exactly!! 😄 You got it!

Put `"Your Construction Site"` first, then put your Manhattan description after it.

And yes, remove the duplicate heading at the end. You only need it once!

You're understanding how Python lists keep things in order.

ok so im done with route function 3?

Almost! 😊 You've worked out the content and structure for Route 3, but I haven't seen your revised code with the heading moved to the beginning and the duplicate removed.

Did you make those two changes? If so, Route 3 is done!

```
@route("/your")
def your(state: State) -> Page:
    return Page(state, [("Your Construction Site"), ("It's a busy day in Manhattan. A bridge nearby has closed down for repair and has caused a heavier traffic jam come rush hour. Design a bridge over the neighboring river to help solve this problem."),
    ])
```

Yes! 🎉 Your Route 3 has the heading first, the Manhattan description second, and no duplicate heading.

One small syntax detail: your trailing comma after the description is allowed in Python, so that isn't a problem.

Route 3 is complete!

Before moving on, what would you like the button at the bottom of this page to say?

Again: next

Got it! Your Next button should take the player to `/choose_materials` or whichever route you want them to visit next.

What is the URL path you chose for your materials page?

wait hold on where am i putting next

You're putting the Next button inside the same `Page` list, right after the Manhattan description.

Remember, the list contains the things the player sees on the page: the heading, the description, and then the button.

Just like Route 1, the button will be a `Button(...)` item in the list. What punctuation do you think goes after the description string to separate it from the button?

a comma and so i did that for the router before that: return Page(state, [("Project Constraints"), ("Starting budget: $600,000"), ("Time duration: 365.0 days"), ("Construction workers: 100"), ("Next"),
all fine?

You're very close! The commas separating the items are correct. 😊

One thing to check: `"Next"` is currently just a string, so it will display the word Next as text. It won't create a clickable button.

Remember the difference:

- **`"Next"` displays text.**
- **`Button("Next", "/your-next-route")` creates a clickable button.**

Question: Do you remember what goes inside the `Button()` parentheses after `"Next"`?

ok so i did: @route("/project")
def project(state: State) -> Page:
    return Page(state, [("Project Constraints"), ("Starting budget: $600,000"), ("Time duration: 365.0 days"), ("Construction workers: 100"), [Button("Next", "/map of city"),
        "Project Constraints"
    ]) and now i've got the grey stuff again

Don't worry! The grey editor highlighting is usually Python telling you that something about the syntax doesn't look right. We can find it together. 😊

I notice two things in what you pasted:

1. **You used square brackets `[...]` around the `Button(...)` and the heading. But those items should be in the same outer list as the other page content.**
2. **`"Project Constraints"` appears twice. You only need it once.**

Let's fix one thing at a time.

First question: Why do you think there's already a pair of square brackets immediately after `Page(state,`? What do those brackets contain?

@route("/project")
def project(state: State) -> Page:
    return Page(state, [("Project Constraints"), ("Starting budget: $600,000"), ("Time duration: 365.0 days"), ("Construction workers: 100"), Button("Next", "/map of city"),
    ]) ohhh cuz i didnt need it i dont have the grey stuff anymore

YES!! 🎉 You got it! You don't need another pair of square brackets because all the page elements belong in the original list after `Page(state,`.

Your Route 2 now has:

- **Project Constraints heading**
- **Starting budget**
- **Time duration**
- **Construction workers**
- **A clickable Next button**

And the grey highlighting is gone!

Route 2 is looking good. 😊

One thing to remember: `/map of city` needs to match the URL path in your map route exactly.

ohh ok well i just put underscores for map_of_city so is that good?

Yes! Exactly. 😊

If your route is:

@route("/map_of_city")
then your button should use:
Button("Next", "/map_of_city")

The URL inside the button must match the route path exactly, including underscores.

You're doing great!

Ok so now use route 4?

Yes! 😊 Let's move on to Route 4: `bridge_building`.

This is where the player starts planning their bridge.

Before we write anything, let's decide what the player should see on this page.

Would you like the heading to be "Design Your Bridge"?

i was confused at first and you just gave me a great idea

Haha, I love that! 😄 Sometimes the best ideas show up while you're figuring something out.

What are you thinking for Route 4? I'll let you lead the way!

not much i wasn't really thinking on this one T_T

That's okay!! 😭 You don't have to have a brilliant idea for every route. We can figure this one out together, one small piece at a time.

Since Route 4 is `bridge_building`, let's start simple: what should the player do on this page? Maybe this is where they begin putting their bridge design together.

Yeah!

Perfect! 😊 Route 4 can introduce the bridge-building stage, before the player chooses their materials and optional features on Route 5.

Let's give Route 4 a heading first. What do you think of "Bridge Design Begins"?

Nah just keep it at Bridge Design?

Bridge Design it is! 😊

Now, what do you think the player should see underneath that heading? A short introduction explaining that they're about to design their bridge?

Yes just that!

Sounds good! We'll keep Route 4 simple: Bridge Design as the heading, followed by a short introduction.

Something like: “Now it's time to start designing your bridge! Think about the materials and features that will help you build a strong, stable bridge.”

You can put both strings inside the `Page` contents list, just like you did on the earlier routes.

Your turn: What URL path would you like to use for this route?

@route("/bridge_building")
def bridge_building(state: State) -> Page:
    return Page(state, [("Bridge Design"), ("After viewing your constraints and where the bridge will be built, now it's time to build it! Be creative, you're the civil engineer here!"), Button("Next", "/choose_materials"),
    ]) hows this

This looks great!! 🎉 You've got all the important pieces in place.

Let's check it together:

- **`@route("/bridge_building")` — matches your route name.**
- **`def bridge_building(state: State) -> Page:` — your function is named consistently.**
- **`"Bridge Design"` — clear page heading.**
- **Your introduction — explains the player's role as the civil engineer.**
- **`Button("Next", "/choose_materials")` — takes the player to the materials page.**

One small thing to think about: Your introduction says “now it's time to build it,” but the next page is where players choose materials. Does “start designing your bridge” describe this stage a little more accurately?

Otherwise, your structure looks consistent with the routes you've written so far.

Fixed that little detail

Nice!! 🎉 Route 4 is in good shape.

You've now got four routes drafted:

1. **Home**
2. **Project Constraints**
3. **Your Construction Site**
4. **Bridge Design**

Next up is Route 5: `choose_materials` — this is where your SelectBox and CheckBoxes come into play.

Ready to tackle it?

Ready as I’ll ever be

Haha, we're getting there! 😂 One page at a time.

Route 5 is where players choose their construction materials and optional features.

Let's start with just the heading. How about "Choose Your Materials"?

Would you like to use that, or do you have another title in mind?

Nope I like that

Perfect! 😊 "Choose Your Materials" it is.

Let's put that heading at the top of Route 5.

Next question: Do you want a short introductory sentence underneath explaining that players can choose materials and optional features for their bridge?

Nope they get the gist of it

Fair enough! No need to repeat information the player already understands. 😄

So Route 5 will have:

- **Choose Your Materials heading**
- **SelectBox for material choices**
- **CheckBoxes for optional features**
- **Remaining budget displayed and updated as choices are made**

Let's start with the SelectBox. What construction materials are you thinking of offering players?

Steel , Concrete , Stone , Asphalt and Aluminum

Ooh, that's a nice variety! 😊 You've got five material options:

1. **Steel**
2. **Concrete**
3. **Stone**
4. **Asphalt**
5. **Aluminum**

Before we work on the SelectBox, one important design question:

Do you want players to choose just one material, or combine multiple materials in a single bridge?

Think about how that affects your `selected_material` attribute in State, too!

well wait when you say affects selected_material is that gonna be a problem if I say they can choose multiple

Not a problem at all!! 😊 It just means we may need to adjust the type of `selected_material`.

Right now, you have:

selected_material: str

A `str` represents one piece of text, so it naturally fits one selected material. If players can choose multiple materials, a list of strings could represent their selections instead.

For example, a list could hold `"Steel"` and `"Concrete"` together.

Your game design doesn't need to change. We'd just make sure the State attribute can represent what the player is allowed to choose.

So, would you prefer players to combine materials in one bridge?

yes- because that's how it'd work in the real world

That makes sense! Bridges often use a combination of materials because different materials serve different purposes.

So we'll change your State attribute from:

selected_material: str
To:
selected_material: list[str]

This tells Python that `selected_material` will hold a list of strings, allowing the player to select Steel, Concrete, Aluminum, or any combination of them.

One other thing to keep in mind: a standard SelectBox may only allow one selection. We'll want to check whether your course's Drafter components support multiple selections before deciding how to build that input.

Next question: Does your assignment or Drafter documentation say whether SelectBox supports multiple selections?

Ok wait hold on- sooo change of plans: instead of selecting multiple things, I'm still gonna use SelectBox, but use it multiple times so that way the player still gets to choose multiple things

Ohhh, yes! That works too! 😊 You can have multiple SelectBoxes on the same page, with each one letting the player choose one material.

That means we can keep your State attribute as:

selected_material: list[str]

Each SelectBox can contribute its selected material to that list. That way, the player can combine materials without needing a multi-select component.

One thing to decide: How many material SelectBoxes do you want players to have? For example, one for each of the five materials, or just a few so they can choose a smaller combination?

The 1st one

Perfect! We'll have five SelectBoxes, one for each material:

1. **Steel**
2. **Concrete**
3. **Stone**
4. **Asphalt**
5. **Aluminum**

Each SelectBox lets the player choose whether to include that material in their bridge. Your `selected_material: list[str]` attribute can keep track of their selections.

That also means we can keep all five SelectBoxes on your Choose Your Materials page.

Next question: Do you want players to be able to leave a SelectBox unselected if they don't want to use that material?

sure, so that way they have some wiggle room if they don't like something

Absolutely! That gives players more flexibility and lets them experiment with different combinations.

So our plan is:

- **Five SelectBoxes, one for each material.**
- **Each SelectBox can be left unselected.**
- **Players can combine whichever materials they want.**
- **`selected_material: list[str]` keeps track of the chosen materials.**

One important detail: leaving a SelectBox unselected should not add anything to the selected materials list.

We'll keep that design for Route 5!

Next, do you want to work on how the material costs affect the remaining budget?

Yes

Great! Let's work through the budget one step at a time.

Your starting budget is $600,000, and you want the remaining budget to decrease whenever a player selects a material.

Since you have five SelectBoxes, each material will need a cost associated with it.

For example, we could use fictional game prices so you can control the difficulty.

First question: Do you want each material to have a fixed cost, or should its cost depend on how much of that material the player uses?

the 2nd one- because even though thats a headache it's important

I actually think that's a worthwhile challenge! It makes the game more realistic because using 10 tons of steel shouldn't cost the same as using 100 tons.

We'll need to account for two things:

- **Material quantity: How much of each material the player uses.**
- **Material cost: How much that quantity costs.**

We can use fictional prices for the game rather than trying to model exact real-world construction estimates.

Let's start with one decision:

Do you want players to enter the amount of each material themselves, or choose from preset amounts (such as small, medium, and large)?

The 1st one

Okay! That gives players even more control over their bridge design.

So each material will have:

- **A SelectBox to choose whether to include it.**
- **A way to enter how much of that material they want to use.**
- **A cost calculated based on the quantity selected.**

For example, a player might choose 20 tons of steel rather than having the game choose the amount for them.

One question: What unit would you like players to use for material quantities: tons, pounds, or something else?

tons because that sounds easier to handle

Tons it is! That keeps the quantities consistent across all five materials. 😊

So our plan for Route 5 is:

- **Five SelectBoxes, one per material.**
- **Each can be left unselected.**
- **Players enter the quantity they want in tons.**
- **The game calculates the cost based on the quantity.**
- **The remaining budget updates from the original $600,000.**

One question: Do you want to use fictional prices per ton for each material, so you can control how expensive each choice is in the game?

Oh yeah definitely

Definitely! That gives you control over how challenging the game is. 😊

We'll give each material its own fictional price per ton. For example, steel could cost more per ton than asphalt.

The important calculation concept is:

Material cost = quantity in tons × price per ton

Then that cost affects the player's remaining budget.

Your textbook explains that Python uses multiplication (*) to calculate products and assignment statements to update variable values.

bakery_complete.md

One question: Would you like to decide the five prices yourself, or would you like help brainstorming balanced fictional prices?

nope i've got it all right here: steel $2,000/ton; concrete: $200,000; asphalt: $4; stone: $45,000; aluminum: $8,500 - now I don’t know if that's close to 600,000 but you know how to do math

Haha, absolutely! 😄 Let's check the numbers.

Material	Your price
Steel	**$2,000/ton**
Concrete	**$200,000/ton**
Asphalt	**$4/ton**
Stone	**$45,000/ton**
Aluminum	**$8,500/ton**

One important detail: I'm treating each price as a per-ton cost, including concrete, asphalt, and stone.

Your prices can work as fictional game prices, but the budget depends on how many tons the player chooses. For example, 3 tons of steel would cost $6,000.

One question: Did you intend concrete to cost $200,000 per ton? That's a very large price compared with the others, so I want to make sure before we build the cost system around it.

you're right too expensive- lets go with 100,000

Perfect! Concrete is now $100,000 per ton.

Your updated fictional prices are:

Material	Price per ton
Steel	**$2,000**
Concrete	**$100,000**
Asphalt	**$4**

Stone	**$45,000**
Aluminum	**$8,500**

Your $600,000 starting budget stays the same.

Next, we'll eventually use each material's quantity in tons multiplied by its price per ton to calculate the cost.

Would you like to move on to deciding the optional feature prices?

Yeah sure they have 3 to pick from: Timber, Rubber & Synthetic Rubber and Fiber-Reinforced Polymers

Nice! Those are your three optional features:

1. **Timber**
2. **Rubber & Synthetic Rubber**
3. **Fiber-Reinforced Polymers**

Since your players have a $600,000 budget, let's give each feature a fictional price that makes choosing them a meaningful decision.

I'd suggest starting with these:

Optional feature	Suggested cost
Timber	**$5,000**
Rubber & Synthetic Rubber	**$15,000**
Fiber-Reinforced Polymers	**$30,000**

These are fictional game costs, not real-world market prices.

Do those prices work for your game, or would you like to adjust any of them?

Nope I like all that!

Awesome! 🎉 Your material and optional feature prices are settled.

Construction materials

Material	Cost per ton

Steel	**$2,000**
Concrete	**$100,000**
Asphalt	**$4**
Stone	**$45,000**
Aluminum	**$8,500**

Optional features

Feature	Cost
Timber	**$5,000**
Rubber & Synthetic Rubber	**$15,000**
Fiber-Reinforced Polymers	**$30,000**

Your starting budget remains $600,000.

One thing at a time: Do you want the optional features to have fixed costs, or should their costs also depend on how many tons the player uses?

2nd one

Got it! The optional features will also be priced per ton, just like the construction materials.

Optional feature	Cost per ton
Timber	**$5,000**
Rubber & Synthetic Rubber	**$15,000**
Fiber-Reinforced Polymers	**$30,000**

That means players will need to enter a quantity for each optional feature they choose.

Next question: Do you want to use the same unit, tons, for all three optional features?

eh yeah use tons, idk what the conversion is for lbs anyway

Sounds good! We'll use tons for everything—both construction materials and optional features. No pound conversions needed.

Your pricing system is now consistent:

- **Construction materials: cost per ton.**
- **Optional features: cost per ton.**
- **Starting budget: $600,000.**

And that gives players flexibility to experiment with different combinations.

One small decision: Should players be allowed to choose zero tons of an optional feature if they don't want to include it?

yep thats also an option

Perfect! Zero tons means the player skips that feature entirely. That keeps the choices flexible.

So we've settled:

- **All materials and optional features are measured in tons.**
- **Each has its own price per ton.**
- **Players can enter zero to skip an option.**
- **All costs come out of the $600,000 starting budget.**

You're making great progress! Next, we can work on how the quantities and prices will interact with your `remaining_budget` attribute in your `State` dataclass.

here comes the not-so-fun part huh

Haha, a little bit! 😂 But I promise we won't tackle everything at once.

You've already done the creative planning. Now we're just teaching Python how to keep track of the choices players make.

Let's start with one small concept:

Your `remaining_budget` starts at $600,000. When a player chooses materials, the cost needs to come out of that amount.

Before we touch the code, what do you think Python should do first: calculate the total cost of the player's selections, or subtract each selection's cost as they make it?

which one do you think is mentally easier for me

I think calculating the total cost first, then subtracting it once will be mentally easier for you.

Here's why:

- **You calculate the cost of each material and optional feature.**
- **Add those costs together.**
- **Subtract that one total from the $600,000 budget.**

It also makes it easier to check whether the player went over budget.

We'll take it one little piece at a time. ❤️

First question: Does it make sense that each material's cost is its quantity in tons multiplied by its price per ton?

Sounds about right

Exactly! You've got it. 😊

For example, if a player chooses 2 tons of steel at $2,000 per ton:

2 x 2000 = 4000

That's the material's total cost.

Your textbook explains that Python uses `*` for multiplication and `+` for addition.

bakery_complete.md

Your turn: If a player chooses 3 tons of aluminum at $8,500 per ton, what expression would you use to calculate its cost? (No need to write Python code yet!)

3 x 8500

Good! You used multiplication: tons × price per ton.

Now, what does `3 * 8500` represent in this example—the quantity, the price per ton, or the total cost?

total cost of that certain material

Exactly! `3 * 8500` calculates the total cost of the aluminum: 3 tons multiplied by $8,500 per ton.

Now let's connect that to your budget. If you calculate the cost of each material separately, what would you need to do with those individual costs to find the overall material cost?

Add em

Yes! Add the individual material costs together to get the total material cost.

Then you can subtract that total from your starting budget of $600,000.

One question: Why do you think subtracting the total just once is easier than subtracting each material cost separately?

so that way I don't tweak out- lets focus on the code please

Absolutely 😂 Let's focus on the code, one small piece at a time.

Since we're working on material costs, let's start with what happens when a player enters a quantity.

Do you already have the five material quantity inputs on your `choose_materials` page, or are we building those next?

I think building?

No worries! Let's build them together. 😊

We'll start with just one material: Steel. Since players need to enter how many tons they want, which Drafter input do you think would let them type a number: `TextBox` or `SelectBox`?

Textbox since it's gonna be something they'll type out

Exactly! A TextBox lets the player type in their quantity.

In Drafter, the input can be stored in a variable in your `State`.

Let's think about the data type: if the player enters `3` tons of steel, should that value be stored as an `int` or a `str`? Why?

Int since 3's a number

Right! `3` is an integer (`int`), so that makes sense.

One small thing to keep in mind: a TextBox initially gives you text (`str`), even when the player types a number. You'll need to convert that text into an integer before doing multiplication.

Do you remember which Python function converts a string into an integer?

Nope- crossed my mind entirely

No worries! That's `int()`.

For example, in a separate little example:

typed_number = "3"

number = int(typed_number)

Now `number` is the integer `3`, rather than the string `"3"`.

Quick check: Why can't we multiply the original string `"3"` directly by a price like `8500` and expect ordinary arithmetic?

because one's a str and one's an int

Exactly! You've got it. Python treats strings and integers as different types, so we convert the input before doing arithmetic.

Now let's connect this to your `State` dataclass.

Do you currently have an attribute in `State` to store the number of tons of steel a player enters?

are you genuinely asking or is that a trick question

Genuinely asking! 😅 Not a trick question, I promise.

I remember your `State` dataclass, but I don't remember you adding an attribute specifically for the tons of steel the player enters.

If you haven't added one yet, that's completely fine. We'll figure out where it belongs together.

alright lets do it then

Let's do it! We'll add one attribute at a time.

Your `State` dataclass already stores information about the game. We need a place to store how many tons of steel the player enters.

What type should that new attribute have: `int`, `str`, or something else? Remember, the player will type the quantity into a TextBox.

str because like u said- it's gonna be in a textbox

Exactly! 😊 The TextBox gives us a `str`, so storing the player's raw input as a string makes sense.

Now, what would you name the attribute in your `State` dataclass? Something like `steel_quantity` would describe what it stores.

quantity_of_steel

`quantity_of_steel` is a clear, descriptive name!

Add it as an attribute in your `State` dataclass, with the type `str`.

What value do you think it should start with before the player enters anything?

“”

Yes! An empty string (`""`) is a good starting value because the player hasn't entered a quantity yet.

So your attribute will start empty.

Next, would you like to add the quantity attribute for concrete?

like- naming it?
Yes! I meant the attribute name. 😊

What would you call the attribute that stores the player's concrete quantity? You can use the same naming pattern as `quantity_of_steel`.

yup exactly that- for the rest of the materials if anything

Perfect! We'll use the same pattern for all five materials:

- `quantity_of_steel`
- `quantity_of_concrete`
- `quantity_of_stone`
- `quantity_of_asphalt`
- `quantity_of_aluminum`

Each will have type `str` and start with `""`.

That gives each TextBox its own place to store the player's input. 😊

Next, are you ready to add the quantity attributes for the optional materials too?

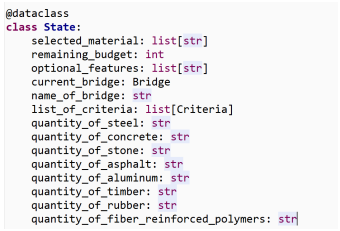
Yep

Great! 😊 The optional materials are:

- `quantity_of_timber`
- `quantity_of_rubber`
- `quantity_of_fiber_reinforced_polymers`

Use `str` for each, with an initial value of `""`, just like the other material quantities.

That gives you eight quantity attributes altogether. Once you've added them, we'll connect them to the TextBoxes on your page.



```
@dataclass
class State:
    selected_material: list[str]
    remaining_budget: int
    optional_features: list[str]
    current_bridge: Bridge
    name_of_bridge: str
    list_of_criteria: list[Criteria]
    quantity_of_steel: str
    quantity_of_concrete: str
    quantity_of_stone: str
    quantity_of_asphalt: str
    quantity_of_aluminum: str
    quantity_of_timber: str
    quantity_of_rubber: str
    quantity_of_fiber_reinforced_polymers: str
```

Nice work! 😊 You've added all eight quantity attributes, and the names follow a consistent pattern.

One important thing to check: your screenshot shows the type annotations, but not the initial values. In your `State` instance, each quantity should start as `""` so the TextBoxes begin empty.

Your next step is connecting these attributes to the TextBoxes on `choose_materials`.

Let's start with steel: do you remember which Drafter component lets the player type text into a page?

Textbox?

Yes! Exactly — a TextBox lets the player type a value. 😊

For the steel quantity, we'll connect that TextBox to `quantity_of_steel`.

Do you remember what argument in Drafter's `TextBox` connects it to a variable in `State`?

```
@route
def index(state: State) -> Page:
    return Page(state, [
        "Current motto: " + state.motto + "\n",
        "A new motto:",
        TextBox("new_motto", state.motto),
        "\n",
        Button("Update", "update")
    ])
```

something like this?

Yes! That's the exact pattern. 😊 Your textbook shows this same approach in 4A4, "Complex State," where it uses:

`TextBox("name", state.name)`

Here's what each part means:

- `"name"` **is the input's name. Drafter uses it to pass the entered text to a route function when a button is clicked.**
- `state.name` **is the current value displayed in the TextBox.**

For your steel quantity, you'd follow that same pattern using your steel attribute.

One question: What do you think the first argument (the text in quotation marks) should be called for the steel TextBox?

"quantity of steel"

That's a reasonable label! 😊 One small naming detail: Drafter's first argument is a string identifier, so it doesn't have to match your State attribute exactly.

Your State attribute is `quantity_of_steel` (with underscores). Would you prefer to use that same name as the TextBox identifier to keep things consistent?

actually you know what no I wanna say amount of steel needed

I like that! "amount of steel needed" is much more player-friendly. 😊

You can use that as the TextBox identifier. Just remember: the identifier and the State attribute serve different purposes.

Your State attribute stays `quantity_of_steel`.

What do you think should go in the second argument of the TextBox so it displays the current value from State?

Update

Almost! I mean the State attribute that holds the current steel quantity.

Think back to your `State` dataclass. What did you name that attribute? 😊

oh quantity_of_steel

Exactly! 😊 So the second argument refers to `state.quantity_of_steel`.

That way, the TextBox displays the current value stored in your State.

You've got the two pieces now:

- Identifier: `"amount of steel needed"`
- Current value: `state.quantity_of_steel`

Next, let's think about what happens when the player clicks a button. How will the typed value get saved back into State?

through textbox?

Yes, the TextBox is where the player types! You're thinking in the right direction. 😊

There's one more piece: the TextBox captures the input, but the button click triggers the route function, where Drafter can update the State.

Do you remember what Drafter calls the value passed from the TextBox into that route function?

@route
def update(state: State, new_motto: str) -> Page:
  state.motto = new_motto
  return index(state) does this have something to do with it?
Yes! That example is directly related. The important part is the route parameter:
def update(state: State, new_motto: str) -> Page:
`new_motto` is the value received by the route from the TextBox. Then this line:
state.motto = new_motto

stores that received value in the State.

For your project, think about the connection: What would you expect the route parameter to be called if the TextBox identifier is `"amount of steel needed"`?

quantity of steel since that's the whole point of that part

That makes sense! `quantity_of_steel` is a clear name for the route parameter because it describes what the TextBox collects.

One important distinction: the TextBox identifier (`"amount of steel needed"`) and the route parameter (`quantity_of_steel`) need to match for Drafter to pass the value correctly.

Can you see how those two names are different in your current setup?

ohhh so both need to be the same name

Exactly! The TextBox identifier and the route parameter need to match so Drafter knows which input value to pass.

So if your State attribute is `quantity_of_steel`, your route parameter can use that same name too.

One small distinction: the State attribute and the route parameter are still separate variables. The route assigns the parameter's value to the State attribute.

Ok? Can we please get on with coding

Absolutely! Let's get back to coding. 😊

We'll work on one small piece at a time. For the steel quantity, you already have the TextBox identifier and State attribute in mind.

Next, let's focus on the route that handles the button click. Do you already have a route for submitting the material choices, or are we creating one?

i only have this part done: @route("/choose_materials")
def choose_materials(state: State) -> Page:
return Page(state, ["Choose Your Materials"),
])

Perfect. We can build from exactly where you are. First, there's a small syntax issue in your current `Page`: the heading string starts with `"` but closes with `)` instead of `"`.

Let's focus on the next piece: the steel quantity TextBox.

In Drafter, a TextBox is added as an element inside the Page's element list. You can use the textbook's example, `TextBox("name", state.name)`, as a reference for its two arguments.

What should replace `"name"` in that example for your steel quantity input?

wait hold on- so would route 5 look something like: @route
def index(state: State) -> Page:
return Page(state, [
"Current motto: " + state.motto + "\n",
"A new motto:",
TextBox("new_motto", state.motto),
"\n",
Button("Update", "update")
])

Yes! That's the same Drafter page structure you can use for route 5. Your materials page will follow that pattern, but with your own heading and input fields.

In your example:

- **`Page(state, [...])` builds the page.**
- **`TextBox("new_motto", state.motto)` creates an input connected to a State attribute.**
- **`Button("Update", "update")` creates a button that navigates to the `"update"` route.**

For your steel quantity, the equivalent TextBox would use the identifier `"quantity_of_steel"` and the State value `state.quantity_of_steel`.

One important distinction: the text players see, such as "Amount of steel needed:", can be separate from the TextBox identifier.

Does that distinction make sense?

yes it does so could I just copy and paste that in there and rename some parts?

Yes! You can use it as a template and rename parts to fit your materials page. Just don't copy it unchanged, because the motto example has different State attributes and a different button destination.

For your page, you'll need to adapt:

- The heading to `"Choose Your Materials"`.
- Each TextBox identifier and State attribute to match the material quantity.
- The button label and route destination to match your next page.

I'd suggest adding one TextBox at a time so it's easier to spot mistakes.

Let's start with steel: what State attribute will hold the player's steel quantity?

"current moto"?

Oh! `"Current motto"` is just text from the textbook's example. It has nothing to do with your bridge project. 😊

You can replace it with something like `"Amount of steel needed:"` so the player knows what to enter. Your TextBox will connect to `state.quantity_of_steel`.

Ok so what do you want me to change

Let's take it one small piece at a time! In your textbook example, change these parts:

1. `"Current motto: " + state.motto` → `"Choose Your Materials"`
2. `"A new motto:"` → `"Amount of steel needed:"`
3. `TextBox("new_motto", state.motto)` → use the identifier `"quantity_of_steel"` and State attribute `state.quantity_of_steel`.
4. `"Update"` → `"Continue"` (or another button label you prefer).
5. `"update"` → the route you want the button to navigate to next.

Start with just the first change: can you replace the motto heading with `"Choose Your Materials"`?

```
@route("/choose_materials")
def choose_materials(state: State) -> Page:
    return Page(state, [
        "Choose Your Materials" + "\n",
        "Amount of steel needed:",
        TextBox("quantity_of_steel", state.quantity_of_steel),
        "\n",
        Button("Update", "update")
    ])
```

You're applying the textbook pattern nicely! Let's look at what you've written.

A few things to think about:

1. Heading: `"Choose Your Materials"` is now your page heading.
2. Steel input: Your TextBox uses `"quantity_of_steel"` and `state.quantity_of_steel`. Those names line up with your State attribute.

3. **Button: `"Update"` and `"update"` are still from the textbook example. You'll eventually want the button to lead to your next route, which is likely your materials-processing route or bridge-testing page, depending on your design.**

One question before we move on: Have you created a route named `update` yet, or do you want this button to lead to a different route?

well it's going to update to a changed amount of steel right?

Exactly! You're thinking about the purpose correctly. 😊

The player enters a new amount of steel, and the page needs to save that new value in `state.quantity_of_steel`.

The important distinction is that `Button("Update", "update")` navigates to a route named `"update"`. It doesn't automatically update the State attribute just because the button says Update.

The next step is understanding how Drafter passes the TextBox value to that route.

Does that distinction make sense?

yeah it does, now what? Because we also need the other materials, right?

Yes! We need inputs for all five main materials:

- **Steel**
- **Concrete**
- **Stone**
- **Asphalt**
- **Aluminum**

Let's add them one at a time, just like steel. Each one will have:

- **A text label telling the player which material they're entering.**
- **A `TextBox` connected to the matching State attribute.**

For example, your steel TextBox uses `state.quantity_of_steel`.

Let's do concrete next: what State attribute did you create to store the quantity of concrete?

quantity_of_concrete

Exactly! That's the State attribute for concrete.

Now, following the same pattern as steel, what do you think the first argument of the concrete `TextBox` should be?

Remember, it's the identifier that connects the input to the route parameter.

Wait, so hold on what would I do for that? Would I just add a comma to add that in there as a second textbox, what do I do??

Yes!! Exactly. 😊 You add another TextBox inside the same `Page` list, separated from the first one by a comma.

Think of the list as holding all the things you want displayed on the page:

1. Steel label
2. Steel TextBox
3. Concrete label
4. Concrete TextBox
5. And so on.

You already have the steel pair.

For concrete, you'll add a new label and a new `TextBox` using `state.quantity_of_concrete`.

Try adding just those two items after the steel TextBox. You don't need another `Page()` or another route!

```
@route("/choose_materials")
def choose_materials(state: State) -> Page:
   return Page(state, [
      "Choose Your Materials" + "\n",
      "Amount of steel needed:",
      TextBox("quantity_of_steel", state.quantity_of_steel),
      "\n",
      Button("Update", "update"),
      "Amount of concrete needed:",
      TextBox("quantity_of_concrete", state.quantity_of_concrete),
      "\n",
      Button("Update", "update"),
      "Amount of stone needed:",
      TextBox("quantity_of_stone", state.quantity_of_stone),
      "\n",
      Button("Update", "update"),
      "Amount of asphalt needed:",
      TextBox("quantity_of_asphalt", state.quantity_of_asphalt),
      "\n",
      Button("Update", "update"),
      "Amount of aluminum needed:",
      TextBox("quantity_of_aluminum", state.quantity_of_aluminum),
      "\n",
      Button("Update", "update"),
   ]) this is probably very wrong isn't it
```

No, it's not very wrong at all! 😊 You've actually done a good job adding all five materials. The labels and TextBoxes are paired up, and each TextBox references the matching State attribute.

There is one important thing to rethink: the five Update buttons.

Right now, every button points to `"update"`. You probably don't need five separate buttons. You want the player to enter all five quantities and then submit them together.

Think about the page like a form: the player fills in all the ingredients before pressing one button.

Your next task: What would make more sense to the player as the label on one button at the bottom of the materials page — `"Update"` or `"Continue"`?

We'll also need to make sure the route receiving that button click saves the entered quantities into State.

i guess continue makes more sense

I agree! Continue tells the player they're done choosing materials and ready for the next step.

So, keep just one `Button("Continue", ...)` at the bottom of the page, after all five quantity TextBoxes.

One thing to think about next: when the player clicks Continue, what information will the next route need to receive from those TextBoxes?

wait check it first: @route("/choose_materials")
```
def choose_materials(state: State) -> Page:
    return Page(state, [
        "Choose Your Materials" + "\n",
        "Amount of steel needed:",
        TextBox("quantity_of_steel", state.quantity_of_steel),
        "\n",
        "Amount of concrete needed:",
        TextBox("quantity_of_concrete", state.quantity_of_concrete),
        "\n",
        "Amount of stone needed:",
        TextBox("quantity_of_stone", state.quantity_of_stone),
        "\n",
        "Amount of asphalt needed:",
        TextBox("quantity_of_asphalt", state.quantity_of_asphalt),
        "\n",
        "Amount of aluminum needed:",
        TextBox("quantity_of_aluminum", state.quantity_of_aluminum),
        "\n",
        Button("Continue", "continue"),
    ])
```

Yes! This is looking much cleaner. 😊

A few things to check:

- **Your five labels are paired with the correct TextBoxes.**
- **Each TextBox uses its matching State attribute.**
- **You now have just one Continue button at the bottom.**
- **The route syntax and overall structure look consistent.**

One important detail for the next step: `"continue"` is the route destination the button will try to navigate to. You'll need a route registered with that URL.

One question: Have you already created the route that the Continue button should lead to?

no, should I just title it what the next route is gonna be? (test_bridge)

Yes! Since your next page is `test_bridge`, using `"test_bridge"` as the button's destination makes sense.

Just remember there are two connected pieces:

- `Button("Continue", "test_bridge")` **tells Drafter where to navigate.**
- `@route("/test_bridge")` **registers the route that handles that destination.**

Can you see how the button's destination and the route URL match?

yeah i do, now what

Now let's build the `test_bridge` route! This page will be where the player submits their material quantities and you can start checking the bridge's cost.

First, let's focus on the route structure, not the calculations yet.

Start with the route decorator and function definition. Remember, the button sends the player to `/test_bridge`.

Question: What do you think the function needs to receive besides `state: State` so it can access the five quantities entered in the TextBoxes?

Not too sure

That's okay! This is a new Drafter concept, so let's break it down.

Remember how you wrote `TextBox("quantity_of_steel", state.quantity_of_steel)`?

The first argument, `"quantity_of_steel"`, is the input's name. When the player clicks Continue, Drafter can pass the entered value to the next route using that name as a parameter.

For example, a small unrelated syntax example:

```
@route("/greet")
def greet(state: State, name: str) -> Page:
```

Here, `name` is a parameter that receives input.

Looking at your five TextBox names, can you guess what five parameters your `test_bridge` function might need?

ohhh yeah all the 5 from before right?

Exactly!! 🎉 All five from your materials page:

- `quantity_of_steel`
- `quantity_of_concrete`
- `quantity_of_stone`
- `quantity_of_asphalt`
- `quantity_of_aluminum`

One more detail: since TextBox input is text, what type do you think each parameter should have in the function definition?

Its type?

Yes! I mean the data type you should write after the colon for each parameter.

Since a `TextBox` collects text from the player, what Python data type represents text?

Hint: It's one of the basic types: `int`, `float`, `str`, or `bool`. 😊

Str

That's right! `str` represents text. 😊

So each of those five parameters will have type `str`.

Next, think about what happens after the route receives those values: How will you save the values into the matching attributes in `state`?

For example, how would you update `state.quantity_of_steel` using the steel parameter?

set it to a variable?

Close! You already have a variable holding the value: the parameter `quantity_of_steel`.

What you want to do is assign that parameter's value to the matching State attribute.

Think about this pattern:

state.some_attribute = some_parameter
So, in your own words, which side of the `=` should be `state.quantity_of_steel`?
Left

Exactly! 🎉 The State attribute goes on the left, and the parameter goes on the right.

That assignment statement tells Python to update the State attribute with the value received from the TextBox.

The textbook describes this in *Variables: Reading and Writing* and explains that assignment statements write values to variables.

bakery_complete.md

Now try applying that same idea to the other four materials. You don't need to show me all five unless you'd like me to check them! 😊

wait but hold on- where am i putting this in my code

Good question! Put those assignment statements inside your `test_bridge` function, after the function definition and before its `return Page(...)` statement.

Think of the order:

1. **The route receives `state` and the five TextBox parameters.**

2. The assignment statements save those five values into the matching State attributes.
3. The route returns the page.

For example, the placement looks like this in an unrelated toy example:

```
@route("/example")
def example(state: State, flavor: str) -> Page:
    state.favorite_flavor = flavor
    return Page(state, ["Example page"])
```

Your turn: Does it make sense why the assignments belong in `test_bridge`, rather than in `choose_materials`?

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    return Page(state,
    ]) hows this look
```

You're very close! The route header and all five assignments are in the right place. You've correctly matched each parameter to its corresponding State attribute.

There's one thing to look at: your `return Page(state, ])` is incomplete. A `Page` needs a list of content to display between the square brackets.

For now, think about what you want the player to see after clicking Continue.

What should the page display at this point—a heading like "Bridge Testing" or something else?

Yes exactly that

Perfect! Then your `Page` content can start with a heading such as `"Bridge Testing"`.

Try adding that as the first item inside the list in `return Page(state, [...])`.

What do you think you want the player to see or do on the Bridge Testing page after the heading?

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    return Page(state, ["Bridge Testing"
    ]) better?
```

Yes! 😊 This is much better. You now have:

- The correct route URL: `"/test_bridge"`.
- All five parameters, each with type `str`.
- Five assignments that save the submitted values into `State`.
- A `Page` that displays the heading `"Bridge Testing"`.

Your closing `])` is also in the right place.

One question: Do you want the Bridge Testing page to calculate the total material cost next, or would you rather add more page content first?

The 1st one

Great! Let's work on calculating the total material cost. We'll take it one concept at a time.

Right now, your five quantities are stored as `str` values because TextBoxes provide text. But material costs are numerical, so we'll need to convert the quantities before doing multiplication.

For example, in an unrelated toy example:

```
quantity_text = "12"
quantity_number = int(quantity_text)
```

The `int()` function converts the string `"12"` into the integer `12`.

First question: Since your quantities are measured in tons, do you expect players to enter whole numbers only, or should they be able to enter decimal amounts too? That determines whether `int()` or `float()` is appropriate.

nah int makes more sense imo

That makes sense! Using `int()` means players will enter whole-number quantities of tons.

Let's think about the calculation. Each material has its own cost per ton.

For steel, for example:

- Quantity: whatever the player enters
- Cost per ton: $2,000

What mathematical operation would you use to calculate the total steel cost from those two values?

*

Exactly! `*` means multiplication in Python.

So, you're thinking about:

quantity of steel × cost per ton = total steel cost

Now, what would you multiply the steel quantity by: `2000` or `2000.0`?

2000

Yes! `2000` is the steel cost per ton. Since both values are whole numbers, multiplying them will give you an integer.

Let's build on that idea.

If you calculate the cost of all five materials separately, what operation would you use to combine those five costs into one total?

+

Exactly! `+` adds the five individual material costs together.

So now you have the pieces of the calculation:

1. **Convert each quantity from `str` to `int`.**
2. **Multiply each quantity by its material's cost per ton.**
3. **Add the five costs together.**

Let's focus on one material first: Can you identify the State attribute that stores the steel quantity?

state.quantity_of_steel

Exactly! `state.quantity_of_steel` is the State attribute that holds the player's steel quantity.

One important detail: it currently stores a string, so when you calculate the cost, you'll need to convert it to an integer.

In an unrelated example, that looks like:

int("12") * 2000

That evaluates to `24000`.

Your turn: What is the cost per ton of concrete? Check your material-cost list.

Lets focus on code and not side problems

Absolutely. We'll stay focused on your code and work directly on the steel calculation first.

You already identified `state.quantity_of_steel`. Since it's a string, the next step is converting it to an integer.

How would you write an expression that converts `state.quantity_of_steel` to an integer?

We'll build the calculation from your actual State attributes, one piece at a time.

quantity_of_steel = "12"
quantity_number = int(quantity_of_steel) something like this?

Yes! You're using the right idea. `int()` converts a string containing a whole number into an integer.

In your project, though, the quantity is stored in `state.quantity_of_steel`, rather than a separate variable named `quantity_of_steel`.

So, what would you put inside `int()` to convert the steel quantity stored in your `State` object?

`state.quantity_of_steel`

Exactly! Now you're identifying the correct State attribute.

The conversion expression would be `int(state.quantity_of_steel)`.

Next, think about the calculation: steel costs $2,000 per ton. What multiplication operator do you use in Python, and what two values need to be multiplied?

Does this have something to do with what I'm gonna write in my code right after?

Yes! That's exactly what you'll use in the code right after your `test_bridge` route saves the TextBox input into `state.quantity_of_steel`.

You're building the steel-cost calculation:

- First, convert the quantity from a string to an integer.
- Then, multiply that integer by the steel cost per ton ($2,000).
- Store the calculated cost in a variable.

You already have the conversion part. What variable name would you like to use to store the steel cost?

total_steel_cost

Good variable name! `total_steel_cost` clearly describes what the value represents.

Now think about the assignment statement you're building:

- Left side: `total_steel_cost` — where the calculated value will be stored.
- Right side: Your converted steel quantity multiplied by the cost per ton.

What number should you use for the cost per ton of steel?

is that already established..?

Yes! We already established that steel costs $2,000 per ton in your project details.

You don't need to establish it again. You can use that value in your multiplication. What would the full expression on the right side of your assignment statement look like?

state.quantity_of_steel = "2000"
total_steel_cost = int(state.quantity_of_steel)

You're close! You've got the conversion part, but there are two things to reconsider:

1. `state.quantity_of_steel = "2000"` changes the player's entered quantity to 2000. You don't want to overwrite their input with the price.

2. `total_steel_cost = int(state.quantity_of_steel)` converts the quantity, but doesn't calculate its cost yet.

Remember: quantity × price per ton = total cost.

Your `total_steel_cost` variable should hold the result of that multiplication.

What value should be multiplied by the converted quantity?

is it a variable that's going to be multiplied

It can be, yes! It doesn't have to be a variable, though. You can multiply by the number `2000` directly.

For example, in an unrelated context:

total_cost = 3 * 50

Here, `3` is the quantity and `50` is the price per item.

For your steel calculation, what represents the quantity, and what represents the price per ton?

`total_steel_cost` = 2000 * quantity_of_steel

Yes! You've got the multiplication structure right. Two small adjustments to think about:

1. Python variable names don't need backticks or quotation marks in your code.
2. Remember that `quantity_of_steel` is a string stored in your State object, so it needs the `int()` conversion we discussed.

You want to make sure you're multiplying the converted quantity, not the original string.

Can you revise your expression using `int(state.quantity_of_steel)`?